Exp 6: To write C program to implement Priority based CPU scheduling algorithm.

a) Priority in Primptive Mode

```
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ vim priority_p_1290.c
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ gcc priority_p_1290.c
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ ./a.out
Enter number of processes: 5
Enter Arrival Time, Burst Time and Priority for P1: 3
4
6
Enter Arrival Time, Burst Time and Priority for P2: 1
5
2
Enter Arrival Time, Burst Time and Priority for P3: 4
1
4
Enter Arrival Time, Burst Time and Priority for P4: 4
5
7
Enter Arrival Time, Burst Time and Priority for P5: 6
1
8

P       AT      BT      PR      CT      WT      TAT
P1      3       4       6       11      4       8
P2      1       5       2       6       0       5
P3      4       1       4       7       2       3
P4      4       5       7       16      7       12
P5      6       1       8       17      10      11

Average Waiting Time = 4.60
Average Turnaround Time = 7.80
Average Completion Time = 11.40
Throughput = 0.36 processes/unit time


Gantt Chart:

| P2 | P2 | P2 | P2 | P2 | P3 | P1 | P1 | P1 | P1 | P4 | P4 | P4 | P4 | P4 | P5 |
0  1  2  3  4  5  6  7  8  9  10  11  12  13  14  15  16lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$
```

```
lab-03-17@lab-03-17-OptiPlex-3280-AIO: ~

#include <stdio.h>

int main() {
    int n, i;
    int at[10], bt[10], rt[10], pr[10];
    int ct[10], wt[10], tat[10], p[10];
    int time = 0, count = 0, highest;
    float avg_wt = 0, avg_tat = 0, avg_ct = 0, throughput;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        p[i] = i + 1;
        printf("Enter Arrival Time, Burst Time and Priority for P%d: ", p[i]);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
        rt[i] = bt[i];   // remaining time
    }

    int gantt[100], g_index = 0;

    // Preemptive Priority Scheduling
    while (count < n) {
        highest = -1;
        int min_pr = 9999;

        for (i = 0; i < n; i++) {
            if (at[i] <= time && rt[i] > 0) {
                if (pr[i] < min_pr) {   // smaller number = higher priority
                    min_pr = pr[i];
                    highest = i;
                }
            }
        }

        if (highest == -1) {
            time++;
        } else {
            gantt[g_index++] = p[highest];
            rt[highest]--;   // execute for 1 unit
            time++;

            if (rt[highest] == 0) {
                count++;
                ct[highest] = time;
                tat[highest] = ct[highest] - at[highest];
-- INSERT --                                                          1,1           Top
```

```
Screenshot captured
You can paste the image from the clipboard.

            time++;

            if (rt[highest] == 0) {
                count++;
                ct[highest] = time;
                tat[highest] = ct[highest] - at[highest];
                wt[highest] = tat[highest] - bt[highest];

                avg_wt += wt[highest];
                avg_tat += tat[highest];
                avg_ct += ct[highest];
            }
        }
    }

    // Calculate averages
    avg_wt /= n;
    avg_tat /= n;
    avg_ct /= n;
    throughput = (float)n / (time - at[0]);

    printf("\nP\tAT\tBT\tPR\tCT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i], at[i], bt[i], pr[i], ct[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f", avg_tat);
    printf("\nAverage Completion Time = %.2f", avg_ct);
    printf("\nThroughput = %.2f processes/unit time\n", throughput);

    // Gantt Chart
    printf("\n\nGantt Chart:\n\n|");

    for (i = 0; i < g_index; i++) {
        printf(" P%d |", gantt[i]);
    }

    printf("\n0");
    for (i = 1; i <= g_index; i++) {
        printf("  %d", i);
    }

    return 0;
}
-- INSERT --                                                    86,1
```

b) Priority in Non primptive Mode

```
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ vim priority_np_1290.c
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ gcc priority_np_1290.c
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ ./a.out
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority for P1: 0
3
4
Enter Arrival Time, Burst Time and Priority for P2: 3
5
3
Enter Arrival Time, Burst Time and Priority for P3: 1
5
2

P       AT      BT      PR      CT      WT      TAT
P1      0       3       4       3       0       3
P2      3       5       3       13      5       10
P3      1       5       2       8       2       7

Average Waiting Time = 2.33
Average Turnaround Time = 6.67
Average Completion Time = 8.00
Throughput = 0.23 processes/unit time


Gantt Chart:

 ------------------------
| P1   | P3   | P2   |
 ------------------------
0      3      8      13lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$
```

lab-03-17@lab-03-17-OptiPlex-3280-AIO: ~

```c
#include <stdio.h>

int main() {
    int n, i;
    int at[10], bt[10], pr[10];
    int ct[10], wt[10], tat[10], p[10];
    int completed[10] = {0};
    int time = 0, count = 0, highest;
    float avg_wt = 0, avg_tat = 0, avg_ct = 0, throughput;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        p[i] = i + 1;
        printf("Enter Arrival Time, Burst Time and Priority for P%d: ", p[i]);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
    }

    // Non-Preemptive Priority Scheduling
    while (count < n) {
        highest = -1;
        int min_pr = 9999;

        for (i = 0; i < n; i++) {
            if (at[i] <= time && completed[i] == 0) {
                if (pr[i] < min_pr) {   // smaller number = higher priority
                    min_pr = pr[i];
                    highest = i;
                }
            }
        }

        if (highest == -1) {
            time++;   // If no process has arrived
        } else {
            time += bt[highest];
            ct[highest] = time;
            tat[highest] = ct[highest] - at[highest];
            wt[highest] = tat[highest] - bt[highest];

            avg_wt += wt[highest];
            avg_tat += tat[highest];
            avg_ct += ct[highest];

            completed[highest] = 1;
```

Screenshot captured
You can paste the image from the clipboard.

```c
    for (i = 0; i < n; i++) {
        printf("--------");
    }

    printf("\n|");

    // Print in execution order (based on completion time)
    int printed[10] = {0};

    for (int k = 0; k < n; k++) {
        int min_ct = 9999, pos = -1;
        for (i = 0; i < n; i++) {
            if (!printed[i] && ct[i] < min_ct) {
                min_ct = ct[i];
                pos = i;
            }
        }
        printf("  P%d   |", p[pos]);
        printed[pos] = 1;
    }

    printf("\n ");

    for (i = 0; i < n; i++) {
        printf("--------");
    }

    printf("\n0");

    int printed2[10] = {0};

    for (int k = 0; k < n; k++) {
        int min_ct = 9999, pos = -1;
        for (i = 0; i < n; i++) {
            if (!printed2[i] && ct[i] < min_ct) {
                min_ct = ct[i];
                pos = i;
            }
        }
        printf("      %d", ct[pos]);
        printed2[pos] = 1;
    }

    return 0;
}
```